Advanced Data Management

Query answering in OBDM

Domenico Fabio Savo

Corso di laurea magistrale
INGEGNERIA INFORMATICA
Dipartimento di Ingegneria Gestionale, dell'Informazione e della Produzione
University of Bergamo

Outline of the course

1. Introduction to propositional logic
2. Introduction to First-order Logic
3. Relational Calculus
4. **I**nformation **I**ntegration **S**ystems (IIS)
5. Logical formalization of IISs
6. Mapping between **Global Schema** e **Data Sources**
7. Incomplete information databases
8. Query answering over IISs
9. Ontology-Based Data Management (OBDM)
10. Description Logic Ontologies
11. Query answering in ontologies
12. **Query answering in OBDM**

Ontology-based Data Management

Query answering in ontology-based data management

In OBDM, we have to face the following difficulties:

- The actual data is stored in external information sources (i.e., databases), and thus its size is typically **very large**.
- The ontology introduces **incompleteness** of information, and we have to do **logical inference**, rather than query evaluation.
- We want to take into account at **runtime** the **constraints** expressed in the ontology.
- We want to answer **complex database-like queries**.
- We may have to deal with **multiple information sources**, and thus face also the problems that are typical of data integration.

Questions that need to be addressed

In the context of ontology-based data management:

- Which is the "right" (user's) **query language**?
- Which is the "right" **ontology language**?
- How can we bridge the **semantic mismatch** between the ontology and the data sources? (in an ontology we have objects of the domain while in a database we have values)

OBDM: user's query language

As already shown, we have two borderline cases:

- Just **classes and properties** of the ontology → **instance checking**
  - They are **quite poor as query languages**: we cannot refer to same object via multiple navigation paths in the ontology, i.e., allow only for a limited form of join, namely chaining.

- Full **SQL** (or equivalently, **first-order logic**)
  - Problem: in the presence of **incomplete information**, query answering becomes **undecidable** (FOL validity).

- **A good tradeoff is to use (unions of) conjunctive queries.**

So, we fix (U)CQ as user's query language, i.e., the query we pose over the OBDM system.

Questions that need to be addressed

In the context of ontology-based data management:

- Which is the "right" (user's) **query language** → **UCQ**
- Which is the "right" **ontology language**?
- How can we bridge the **semantic mismatch** between the ontology and the data sources? (in an ontology we have objects of the domain while in a database we have values)

The DL-Lite Family

DL-Lite is a family of DLs optimized according to the **tradeoff** between **expressive power** and **complexity** of query answering, with emphasis on **data**.

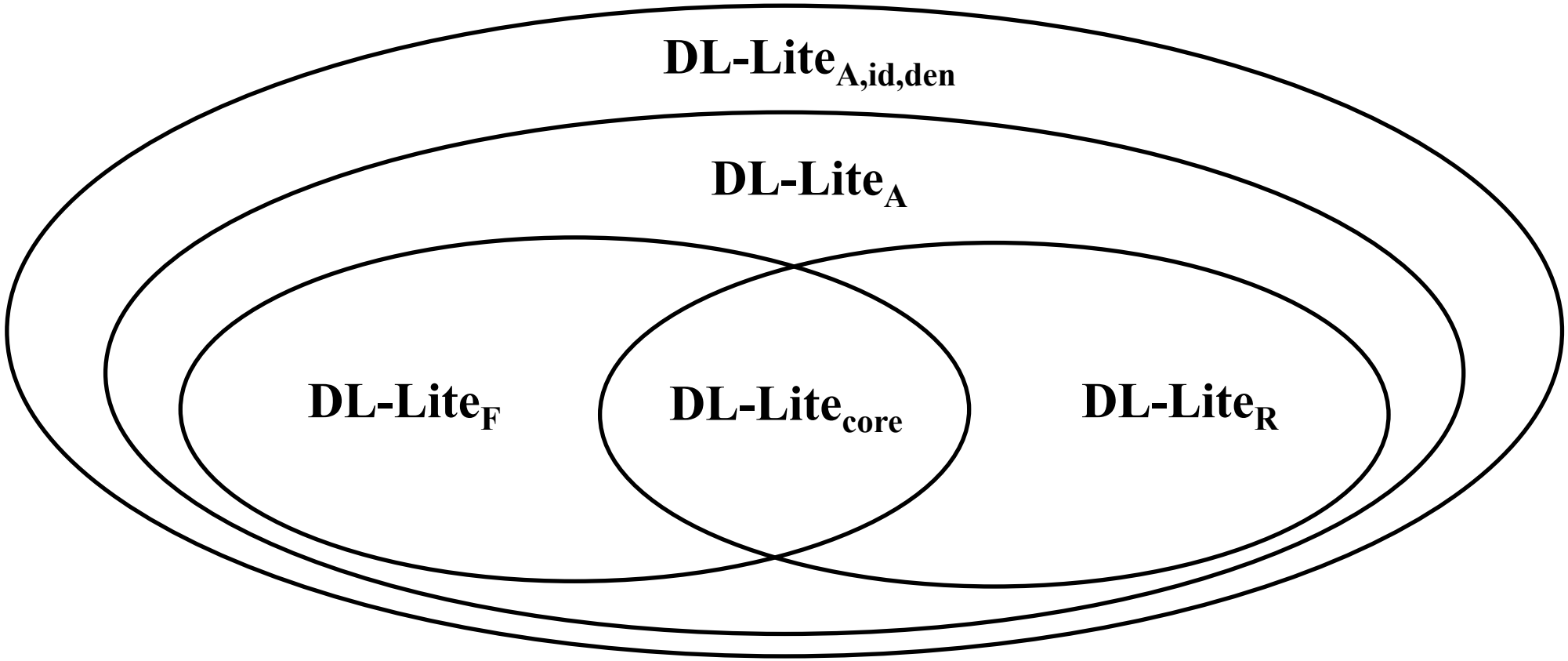
Carefully designed to have **nice computational properties** for answering UCQs (i.e., computing certain answers):

- The same data complexity as **relational databases**.
- In fact, query answering can be delegated to a **relational DB engine**.
- The DLs of the DL-Lite family are essentially the **maximally expressive ontology languages** enjoying these nice computational properties.
- Captures **conceptual modeling formalism**.

DL-Lite provides robust foundations for Ontology-Based Data Management.

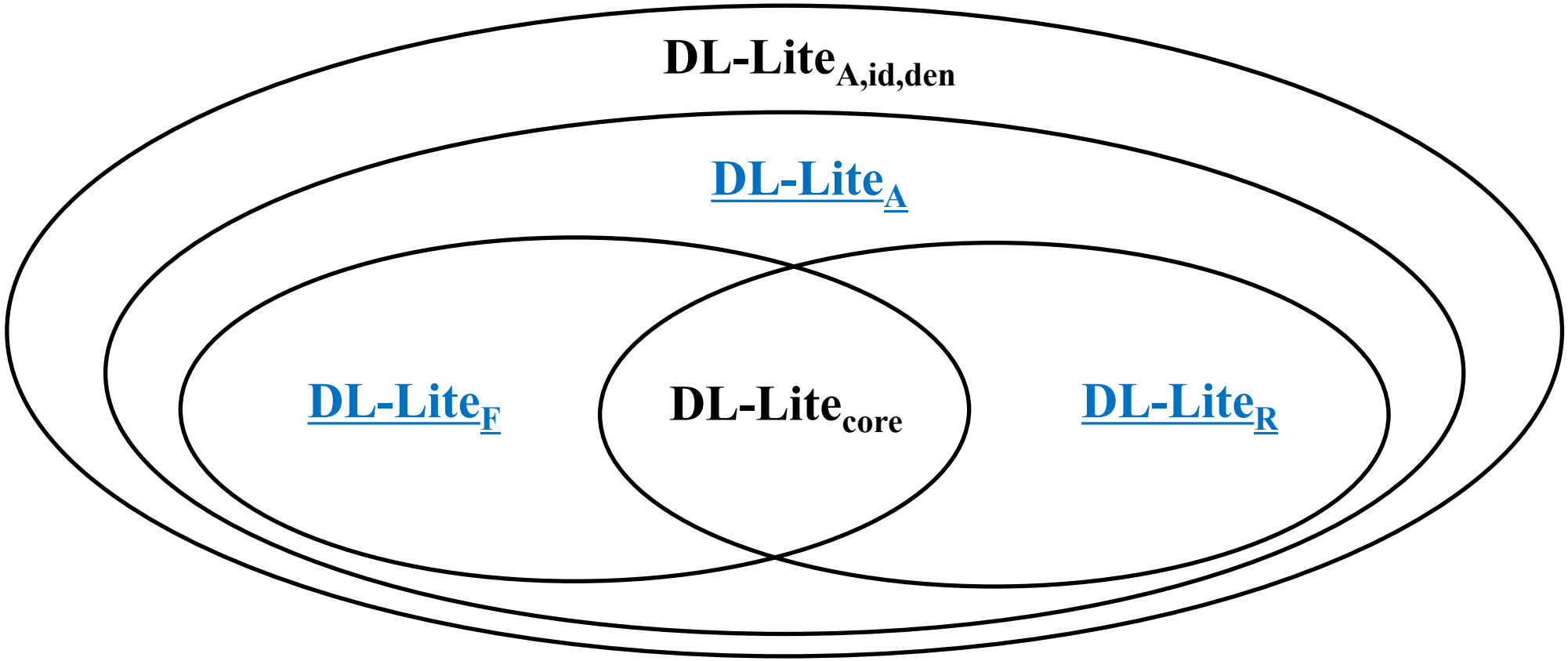
Members of the DL-Lite Family

The main members of the **DL-Lite** is a family are:



Members of the DL-Lite Family

We focus on the following:



Basic features of DL-Lite$_A$

DL-Lite$_A$ is an expressive member of the **DL-Lite family**

- Takes into account the distinction between **objects** and **values**:
  - Objects are elements of an abstract interpretation domain (the instances of concepts).
  - Values are elements of concrete data types, such as integers, strings, ecc. Values are connected to objects through **attributes**.
- Captures most of **UML** class diagrams and **ER** diagrams.
- Enjoys **nice computational properties**, both w.r.t. the traditional reasoning tasks, and w.r.t. query answering (see later).

Syntax of the DL-Lite$_A$ description language

Role expressions:

- atomic role: $\boldsymbol{P}$
- basic role: $\boldsymbol{Q} ::= \boldsymbol{P} \mid \boldsymbol{P^-}$ (where P^- denotes the inverse of the atomic role P)

Concept expressions:

- atomic concept: $\boldsymbol{A}$
- basic concept: $\boldsymbol{B} ::= \boldsymbol{A} \mid \exists \boldsymbol{Q} \mid \boldsymbol{\delta(U)}$
  (where $\exists Q$ denotes the *domain* of the basic role Q, i.e., set of objects participating to Q as first element, while $\delta(U)$ denotes the *domain* of the attribute U, i.e., the set of objects that U relates to values)

Attribute expressions:

- atomic attribute: $\boldsymbol{U}$

Value-domain expressions:

- attribute range: $\boldsymbol{\rho(U)}$ (i.e., the set of values that the attribute U relates to objects)
- (RDF) datatypes: $\boldsymbol{T_i}$ (such as xsd:string, xsd:integer, xsd:dateTime, etc.)

Concept constructors

Construct	Syntax	Example	Semantics
Atomic concept	A	Student	$A^I \subseteq \Delta_O^I$
Atomic role	P	LivesIn	$\mathrm{P}^I \subseteq \Delta_O^I \times \Delta_O^I$
Atomic attribute	U	Salary	$U^I \subseteq \Delta_O^I \times \Delta_V^I$
Datatype	T_i	xsd:integer	$T_i^I \subseteq \Delta_V^I$ (predefined)
Inverse role	P^-	LivesIn$^-$	$\{(\mathrm{o}', \mathrm{o}) \mid (o, o') \in \mathrm{P}^I\}$
Exis. restriction	$\exists R$	$\exists$LivesIn	$\{ o \mid \exists o'.(o, o') \in R^I\}$
Attribute domain	$\delta(U)$	δ(salary)	$\{o \mid \exists v.(o, v) \in U^I\}$
Attribute range	$\rho(U)$	ρ(salary)	$\{v \mid \exists o.(o, v) \in U^I\}$

Where $\boldsymbol{\Delta_O^I}$ denotes the domain of objects, while $\boldsymbol{\Delta_V^I}$ denotes the domain of values.

DL-Lite$_A$ assertions

TBox assertions can have the following forms:

Inclusion assertions (a.k.a. positive inclusions)

Concept inclusion	$B_1 \sqsubseteq B_2$	Attribute inclusion	$U_1 \sqsubseteq U_2$
Role inclusion	$Q_1 \sqsubseteq Q_2$	Value-domain inclusion	$\rho(U) \sqsubseteq T_i$

Disjointness assertions (a.k.a negative inclusions)

Concept disjointness	$B_1 \sqsubseteq \neg B_2$	Attribute disjointness	$U_1 \sqsubseteq U_2$
Role disjointness	$P_1 \sqsubseteq P_2$		

Functionality assertions

Role functionality	$(\mathbf{funct\ Q})$	Attribute funtionality	$(\mathbf{funct\ U})$

ABox assertions (a.k.a. membership assertions) have the following forms:

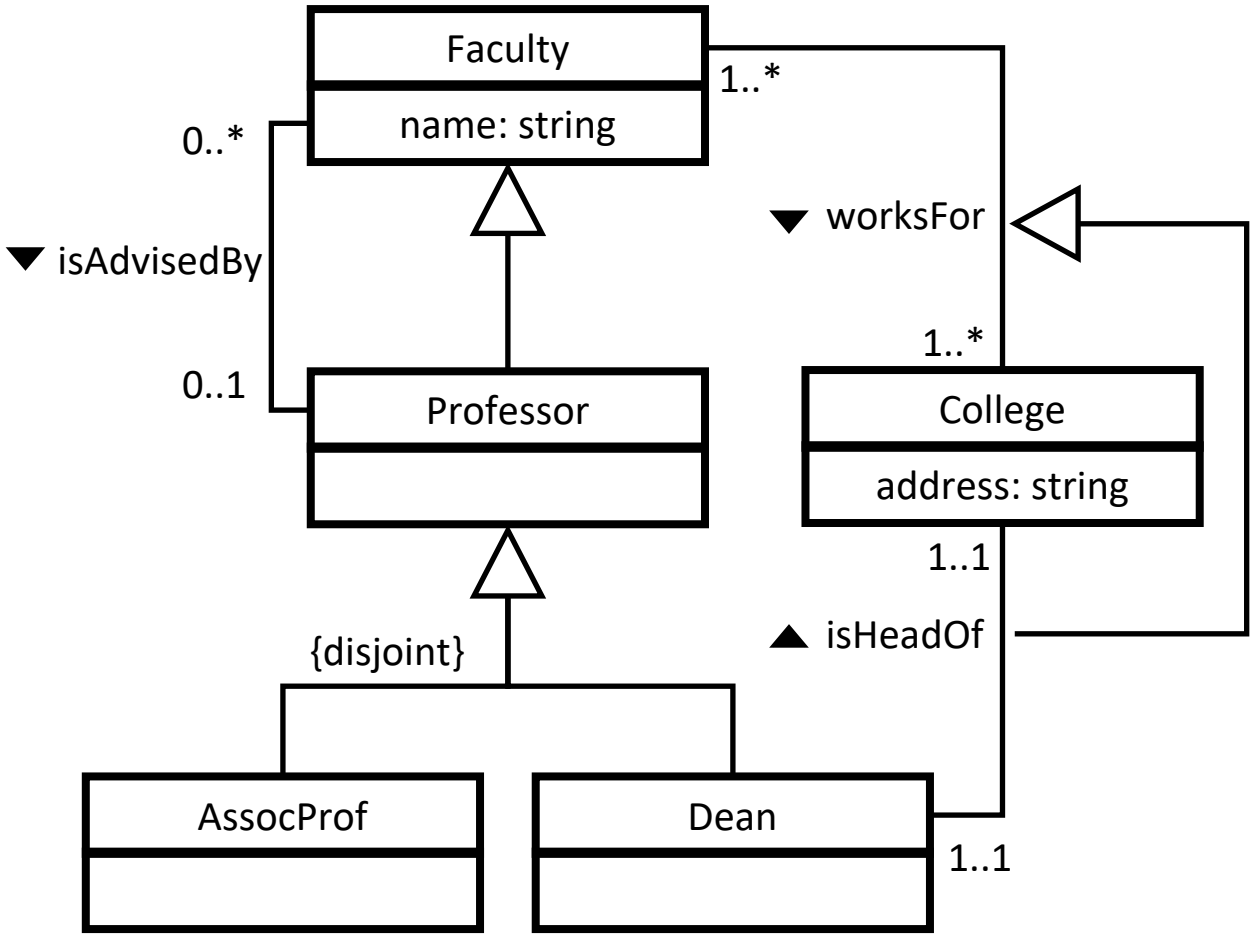
A(c) **P(c,c')** **U(c,v)**

where **c**, **c'** are object constants and **v** is a value constant.

Semantics of a DL ontology

	Syntax	Semantics	Example
Concept inclusion	$\mathbf{B_1 \sqsubseteq B_2}$	$B_1^I \subseteq B_2^I$	Person ⊑ ∃livesIn
Role inclusion	$\mathbf{Q_1 \sqsubseteq Q_2}$	$Q_1^I \subseteq Q_2^I$	hasFather ⊑ hasParent
Attribute inclusion	$\mathbf{U_1 \sqsubseteq U_2}$	$U_1^I \subseteq U_2^I$	cellNumber ⊑ contactInfo
Value-domain inclusion	$\boldsymbol{\rho(U) \sqsubseteq T_i}$	$\left(\rho(U)\right)^I \subseteq T_i$	ρ(name) ⊑ xsd:string
Concept disjointness	$\mathbf{B_1 \sqsubseteq \neg B_2}$	$B_1^I \cap B_2^I = \emptyset$	Car ⊑ ¬Bike
Role disjointness	$\mathbf{P_1 \sqsubseteq P_2}$	$P_1^I \cap P_2^I = \emptyset$	hasFather ⊑ ¬hasMother
Attribute disjointness	$\mathbf{U_1 \sqsubseteq U_2}$	$U_1^I \cap U_2^I = \emptyset$	offPhone ⊑ ¬homePhone
Role functionality	$\mathbf{(funct\ Q)}$	$\forall o, o_1, o_2 . (o . o_1) \in Q^I \wedge (o, o_2) \in Q^I \rightarrow o_1 = o_2$	(funct hasFather)
Attribute funtionality	$\mathbf{(funct\ U)}$	$\forall o, v_1, v_2 . (o . v_1) \in U^I \wedge (o, v_2) \in U^I \rightarrow v_1 = v_2$	(funct SSN))
Con. membership asser.	$\mathbf{A(c)}$	$c^I \in A^I$	Person(bob)
Role. membership asser.	$\mathbf{P(c_1, c_2)}$	$(c_1^I, c_2^I) \in P^I$	hasParent(bob,sam)
Attr. membership asser.	$\mathbf{U(c, v)}$	$\left(c^I, val(v)\right) \in U^I$	name(bob, 'bob')

Example of DL-Lite$_A$ TBox

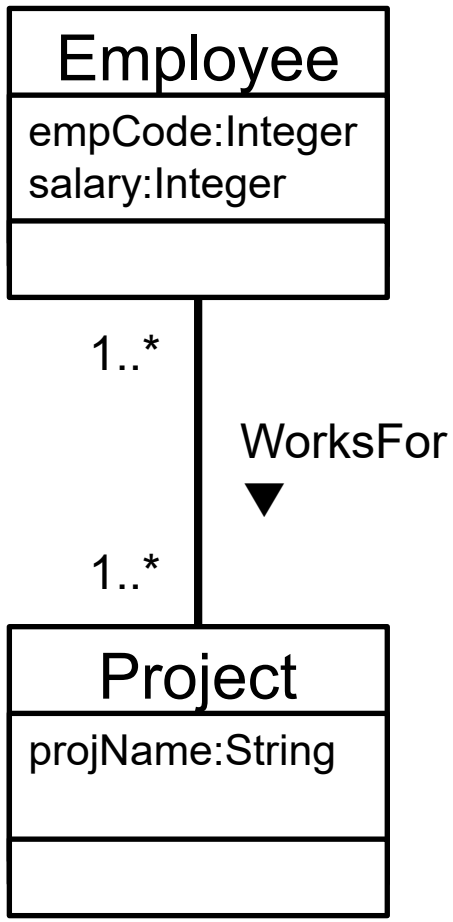


$\mathsf{AssocProf} \sqsubseteq \mathsf{Professor}$
$\mathsf{Dean} \sqsubseteq \mathsf{Professor}$
$\mathsf{AssocProf} \sqsubseteq \neg\mathsf{Dean}$
$\mathsf{Professor} \sqsubseteq \mathsf{Faculty}$
$\mathsf{Dean} \sqsubseteq \exists\mathsf{isHeadOf}$
$\mathsf{College} \sqsubseteq \exists\mathsf{isHeadOf}^-$
$\exists\mathsf{isHeadOf} \sqsubseteq \mathsf{Dean}$
$\exists\mathsf{isHeadOf}^- \sqsubseteq \mathsf{College}$
$\mathsf{Faculty} \sqsubseteq \exists\mathsf{worksFor}$
$\mathsf{College} \sqsubseteq \exists\mathsf{worksFor}^-$
$\exists\mathsf{isAdvisedBy} \sqsubseteq \mathsf{Faculty}$
$\exists\mathsf{isAdvisedBy}^- \sqsubseteq \mathsf{Professor}$
$\exists\mathsf{worksFor} \sqsubseteq \mathsf{Faculty}$

$\exists\mathsf{worksFor}^- \sqsubseteq \mathsf{College}$
$\mathsf{isHeadOf} \sqsubseteq \mathsf{worksFor}$
$\mathsf{AssocProf} \sqsubseteq \neg\mathsf{Dean}$
$\mathsf{Faculty} \sqsubseteq \delta(\mathsf{name})$
$\rho(\mathsf{name}) \sqsubseteq \mathsf{string}$
$\delta(\mathsf{name}) \sqsubseteq \mathsf{Faculty}$
$\mathsf{College} \sqsubseteq \delta(\mathsf{address})$
$\rho(\mathsf{address}) \sqsubseteq \mathsf{string}$
$\delta(\mathsf{address}) \sqsubseteq \mathsf{College}$
(funct isAdvisedBy)
(funct isHeadOf)
(funct isHeadOf$^-$)

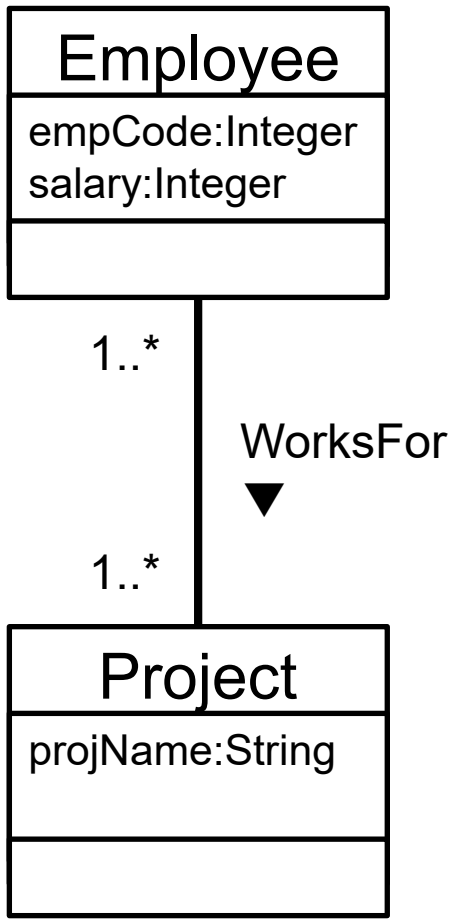
NOTE: DL-Lite **cannot** capture **completeness** of a hierarchy. This would require disjunction (OR).

DL-Lite$_A$ TBox: Exercise



Model the UML diagram with a DL-Lite$_A$ TBox

DL-Lite$_A$ TBox: Exercise – Solution



Model the UML diagram with a DL-Lite$_A$ TBox

$Employee \sqsubseteq \exists WorksFor$
$Project \sqsubseteq \exists WorksFor^-$
$\exists WorksFor \sqsubseteq Employee$
$\exists \mathrm{WorksFor}^- \sqsubseteq Project$
$Employee \sqsubseteq \delta(empCode)$
$Employee \sqsubseteq \delta(salary)$
$Project \sqsubseteq \delta(projName)$
$\delta(empCode) \sqsubseteq Employee$
$\delta(empCode) \sqsubseteq Employee$
$\delta(projName) \sqsubseteq Project$
$\rho(empCode) \sqsubseteq xsd{:}\,integer$
$\rho(salary) \sqsubseteq xsd{:}\,integer$
$\rho(projName) \sqsubseteq xsd{:}\,string$
$(funct\ empCode)$
$(funct\ salary)$
$(funct\ projName)$

Restrictions and assumpions on DL-Lite

To ensure the good computational properties that we aim at, we have to:

Impose a **restriction** on the use of **functionality** and role/attribute inclusions.

- **Restriction on DL-Lite$_A$ TBoxes:** No functional role or attribute can be specialized by using it in the right-hand side of a role or attribute inclusion assertion, that is if (funct P) or (funct P^-) is in **T**, then $Q \sqsubseteq P$ and $Q \sqsubseteq P^-$ are cannot be **T** and If (funct U) is in **T** , then $\mathrm{U}' \sqsubseteq U$ cannot be **T**

Make some **assumptions** on how **individuals** are interpreted.

- **Unique name assumption (UNA):** When c_1 and c_2 are two individuals such that $c_1 \neq c_2$ then $c_1^I \neq c_2^I$.
  Note that when the **UNA** holds (as in DL-Lite), **functionality** cannot be used to derive new information, they can be considered as **constraints**

DL-Lite$_F$ and DL-Lite$_R$ of the DL-Lite Family

We consider also two sub-languages of **DL-Lite$_A$** (that trivially obey the previous restriction):

- **DL-Lite$_F$:** Allows for functionality assertions, but does not allow for role inclusion assertions.
- **DL-Lite$_R$:** Allows for role inclusion assertions, but does not allow for functionality assertions.

In both **DL-Lite$_F$** and **DL-Lite$_R$** we **do not** consider **data values** (and hence drop value domains and attributes).

Note: in what follows, we will simply use **DL-Lite** to refer to any of the logics of the **DL-Lite family.**

Capturing basic ontology constructs in DL-Lite$_A$

ISA between classes	$A_1 \sqsubseteq A_2$
Disjointness between classes	$A_1 \sqsubseteq \neg A_2$
Mandatory participation to relations	$A \sqsubseteq \exists P$ $\quad$ $A \sqsubseteq \exists P^-$
Domain and range of relations	$\exists P \sqsubseteq A$ $\quad$ $\exists P^- \sqsubseteq A$
Functionality of relations	$(funct\ P)$ $\quad$ $(funct\ P^-)$
ISA between relations	$Q_1 \sqsubseteq Q_2$
Disjointness between relations	$Q_1 \sqsubseteq \neg Q_2$
Domain and range of attributes	$\delta(U) \sqsubseteq A$ $\quad$ $\rho(U) \sqsubseteq T_i$
Mandatory and functional attributes	$A \sqsubseteq \delta(U)$ $\quad$ $(funct\ U)$

Properties of DL-Lite

The TBox may contain **cyclic dependencies** (which typically increase the computational complexity of reasoning).

Example: $\boldsymbol{A \sqsubseteq \exists P}$ $\quad$ $\boldsymbol{\exists P \sqsubseteq A}$

In the syntax, we have not included $\sqcap$ on the right hand-side of inclusion assertions, but it can trivially be added, since $\boldsymbol{B \sqsubseteq C_1 \sqcap C_2}$ is equivalent to $\boldsymbol{B \sqsubseteq C_1}$; $\quad$ $\boldsymbol{B \sqsubseteq C_2}$

A **domain** assertion on role P has the form: $\quad \boldsymbol{\exists P \sqsubseteq A}$

A **range** assertion on role P has the form: $\quad \boldsymbol{\exists P^{-} \sqsubseteq A}$

Observations on DL-Lite$_A$

- Captures all the basic constructs of **UML Class Diagrams** and of the **ER Model** . . .

  . . . **except covering constraints** in generalizations.

- Extends (the DL fragment of) the ontology language **RDFS**.

- Is completely symmetric w.r.t. **direct and inverse properties**.

- Is at the basis of the **OWL 2 QL** profile of **OWL 2**.

Complexity of reasoning in DL-Lite$_A$

As shown, DL-Lite$_A$ captures the main features of main **conceptual modeling formalisms**.

We will show that DL-Lite is characterized by the following computational properties:

- **Ontology satisfiability** and all classical DL reasoning tasks are:
  - Efficiently tractable in the size of the **TBox** (i.e., **PTime**).
  - Very efficiently tractable in the size of the **ABox** (i.e., **AC^0**).
- **Query answering** for CQs and UCQs is:
  - **PTime** in the size of the **TBox**.
  - **AC^0** in the size of the **ABox**.
  - Exponential in the size of the **query** (**NP-complete**).
  - **Bad? . . . not really, this is exactly as in relational DBs.**

DL-Lite is essentially the maximal DL enjoying these nice computational properties.

From the observations above we get that: DL-Lite is a DL very well suited to underlie **ontology-based data management systems!**

Questions that need to be addressed

In the context of ontology-based data management:

- Which is the "right" (user's) **query language** → **UCQ**
- Which is the "right" **ontology language** → **DL-Lite**
- How can we bridge the **semantic mismatch** between the ontology and the data sources? (in an ontology we have objects of the domain while in a database we have values)

Reasoning and query answering in DL-Lite

TBox and ABox reasoning services

- Ontology Satisability: Verify whether an ontology **O** is satisfiable, i.e., whether **O** admits at least one model.
- Concept Instance Checking: Verify whether an individual **c** is an instance of a concept **C** in an ontology **O**, i.e., whether **O** $\models$ **C(c)**.
- Role Instance Checking: Verify whether a pair **(c, c')** of individuals is an instance of a role **R** in an ontology **O**, i.e., whether **O** $\models$ **R(c, c')**.
- Query Answering Given a query **q** over an ontology **O**, find all tuples $\bar{c}$ of constants such that **O** $\models$ **q($\bar{c}$)**.

Query answering and instance checking

For **atomic concepts and roles**, **instance checking is a special case of query answering**, in which the query is **Boolean** and constituted by a single atom in the body.

- **O** $\models$ **A(c)** iff **q =** $\{(\quad) \mid A(c)\}$ evaluated over **O** is *true*.
- **O** $\models$ **P(c, c')** iff **q =** $\{(\quad) \mid P(c, c')\}$ evaluated over **O** is *true*.

From instance checking to ontology unsatisfiability

Theorem: Let **O** = ⟨**T**, **A**⟩ be a **DL-Lite** ontology, C a DL-Lite concept, and P an atomic role. Then:

- $\boldsymbol{O} \models \boldsymbol{C}(\boldsymbol{c})$ iff $\boldsymbol{O}_{\boldsymbol{C}(\boldsymbol{c})} = \langle \mathbf{T} \cup \{ \widehat{\boldsymbol{C}} \sqsubseteq \neg \boldsymbol{C} \}, \ \mathbf{A} \cup \{\widehat{\boldsymbol{C}}(\boldsymbol{c})\} \rangle$ is **unsatisfiable**, where $\widehat{\boldsymbol{C}}$ is a new atomic concept not appearing in **O**.
- $\boldsymbol{O} \models \boldsymbol{P}(\boldsymbol{c}, \boldsymbol{c}')$ iff $\boldsymbol{O}_{\boldsymbol{P}(\boldsymbol{c},\boldsymbol{c}')} = \langle \mathbf{T} \cup \{ \widehat{\boldsymbol{P}} \sqsubseteq \neg \boldsymbol{P} \}, \ \mathbf{A} \cup \{\widehat{\boldsymbol{P}}(\boldsymbol{c}, \boldsymbol{c}')\} \rangle$ is **unsatisfiable**, where $\widehat{\boldsymbol{P}}$ is a new atomic role not appearing in **O**.

Theorem: Let **O** = ⟨**T**, **A**⟩ be a **DL-Lite$_F$** ontology and **P** an atomic role. Then **O** ⊨ **P(c, c')** iff the ontology **O** is **unsatisfiable** or if **P(c, c')** ∈ **A**.

Certain answers

We recall that:

Query answering over an ontology **O** = ⟨**T**, **A**⟩ is a form of **logical implication**: find all tuples $\bar{c}$ of constants such that **O** ⊨ **q**($\bar{c}$).

a.k.a. **certain answers** in incomplete databases, i.e., the tuples that are answers to **q** in all models of **O** = ⟨**T**, **A**⟩

$$cert(q, O) = \{\bar{c} \mid \bar{c} \in q^I \; for \; every \; models \; I \; of \; O\}$$

Note: we assume that the answer q^I to a query q over an interpretation I is constituted by a set of tuples of constants of **A**, rather than objects in Δ^I .

FOL-rewritability for DL-Lite

We now study **rewritability** of query answering over **DL-Lite** ontologies.

In particular we will show that **DL-Lite$_A$** (and hence **DL-Lite$_F$** and **DL-Lite$_R$**) enjoy **FOL-rewritability** of answering union of conjunctive queries (UCQ).

That is:

Given an ontology **O** in **DL-Lite** answering UCQ is **FOL-rewritable**, i.e., for every TBox **T** in **DL-Lite** and for every UCQ **q**, the **perfect reformulation** $r_{q,T}$ of **q** w.r.t. **T** can be expressed as a query in **FOL** (and so in **SQL**).

Query answering vs. ontology satisfiability

In the case in which an ontology is **unsatisfiable**, according to the "***ex falso quod libet***" principle, **reasoning is trivialized**.

In particular, **query answering is meaningless**, since every tuple is in the answer to every query.

Clearly, we are **not interested** in encoding meaningless query answering into the perfect reformulation of the input query. Therefore, before query answering, we will always check ontology satisfiability to single out meaningful cases.

Thus, we proceed as follows:

1. We show how to do **query answering over satisfiable ontologies**.
2. We show how we can exploit the query answering algorithm also to **check ontology satisfiability.**

Positive vs. negative inclusions

We call **positive inclusions (PIs)** assertions of the form

$$A_1 \sqsubseteq A_2 \qquad \exists Q \sqsubseteq A \qquad Q_1 \sqsubseteq Q_2$$
$$A \sqsubseteq \exists Q \qquad \exists Q_1 \sqsubseteq \exists Q_2$$

Given a **TBox T**, we denote by $\mathbf{T_P}$ the subset of **T** containing only the **PIs** in **T**.

We call **negative inclusions (NIs)** assertions of the form

$$A_1 \sqsubseteq \neg A_2 \qquad \exists Q \sqsubseteq \neg A \qquad Q_1 \sqsubseteq \neg Q_2$$
$$A \sqsubseteq \neg \exists Q \qquad \exists Q_1 \sqsubseteq \exists \neg Q_2$$

Query answering over satisfiable ontologies

Given a CQ **q** and a **satisfiable** ontology **O** = ⟨**T**, **A**⟩ we compute **cert(q,O)** as follows:

1. By using the TBox **T** (only), we **rewrite** **q** into a UCQ $r_{q,T}$ (the **perfect rewriting** of **q** w.r.t. **T**).
2. We **evaluate** $r_{q,T}$ over **A** (simply viewed as data), to return **cert(q,O)**

Correctness of this procedure shows **FOL-rewritability** of query answering in DL-Lite.

Query rewriting step: Basic idea

Intuition: a **positive inclusion** in the TBox corresponds to a **logic programming rule**.

Basic rewriting step: When an atom in the query unifies with the head of the rule, generate a new query by substituting the atom with the body of the rule. In this case, we say that the positive inclusion **applies** to the atom.

Example: The positive inclusion **AssistantProf ⊑ Professor** corresponds to the logic programming rule **Professor(z) ← AssistantProf(z).**

Consider the input query **q = {(x) | Professor(x)}.**

By applying the positive inclusion to the atom **Professor(x)** in the query, we generate the new query:

- q_1 **= {(x) | AssistantProf(x)}.**

This query is **added** to the input query, and contributes to the perfect rewriting.

Query rewriting – Example 1

Consider the input query

$$\mathbf{q = \{(x) \mid \exists y.teaches(x, y) \wedge Course(y)\}}$$

and the PI: $\exists$**tehacher**$^{-} \sqsubseteq$ **Course** as the logic programming rule:

$$\mathbf{Course(z_2) \leftarrow teaches(z_1, z_2)}$$

The PI applies to the atom **Course(y)** of the input query, so we add to the perfect rewriting the query:

$$\mathbf{q' = \{(x) \mid \exists y, z_1.teaches(x, y) \wedge teaches(z_1, y)\}}$$

Query rewriting – Example 2

Consider the input query

$$\mathbf{q = \{(x) \mid \exists y.teaches(x, y)\}}$$

and the PI: **Professor** $\sqsubseteq$ $\exists$**teaches** as the logic programming rule:

$$\mathbf{teaches(z, f(z)) \leftarrow Professor(z)}$$

Where $f(x)$ is a **Skolem term**.

The PI applies to the atom **teaches**(x, y) of the input query, so we add to the perfect rewriting the query:

$$\mathbf{q' = \{(x) \mid Professor(x)\}}$$

Query rewriting – Example 3

Consider the input query

$$\mathbf{q = \{(x) \mid \exists y.teaches(x, database)\}}$$

and the same PI: **Professor** $\sqsubseteq$ **$\exists$teaches** as the logic programming rule:

$$\mathbf{teaches(z, f(z)) \leftarrow Professor(z)}$$

The atom **teaches(x, database)** **does not unify** with **teaches(z, $f(z)$)** since the **Skolem term** $f(z)$ in the head of the rule **does not unify** with the constant **database**

In this case, we say that the PI **does not apply** to the atom **teaches(x,database)**, so, in this case, we do not add any new query to the perfect rewriting.

Query rewriting – Example 4

Consider the input query (note that y is now a **distinguished** variable)

$$\mathbf{q = \{(x, y) \mid teaches(x, y)\}}$$

and the same PI: **Professor** $\sqsubseteq$ $\exists$**teaches** as the logic programming rule:

$$\mathbf{teaches(z, f(z)) \leftarrow Professor(z)}$$

Also in this case we have **no unification**. Indeed unifying the **Skolem term** $f(z)$ the variable y would correspond to returning a **Skolem term** as answer to the input query.

Query rewriting – Example 5 (join variables)

An analogous behavior to the one with **constants** and with **distinguished variables** holds when the atom contains **join variables** that would have to be unified with Skolem terms.

Consider the input query:

$$\mathbf{q = \{(x) \mid \exists y.teaches(x, y) \wedge Course(y)\}}$$

and the same PI: **Professor** $\sqsubseteq$ $\exists$**teaches** as the logic programming rule:

$$\mathbf{teaches(z, f(z)) \leftarrow Professor(z)}$$

The PI above does **not apply** to the atom **teaches**(x, y).

Query rewriting – The Reduce step

Consider the query: **q' = {(x) | $\exists y, z_1$.teaches(x, y) $\wedge$ teaches(z_1, y)}**
and the same PI: **Professor $\sqsubseteq$ $\exists$teaches** as the logic programming rule:

teaches(z, $f(z)$) ← Professor(z)

As we saw, the PI above does not apply to **teaches(x, y)** or **teaches(x, y)**, since y is in **join**, and we would again introduce the **Skolem term** in the rewritten query. However, we can transform the above query by **unifying** the atoms **teaches(x, y)** and **teaches(z, y)**. This rewriting step is called **reduce**, and produces the query

q' = {(x) | teaches(x, y)}

Now, we can apply the PI above, and add to the rewriting the query

q'' = {(x) | Professor(x)}

Query rewriting – the procedure (1\2)

To compute the perfect rewriting of a UCQ **q**, start from **q**, iteratively get a CQ **q'** to be processed, and do one of the following steps:

- Apply to some atom of **q'** a PI in **T** as follows:

$$
\begin{array}{rlcl}
A_1 \sqsubseteq A_2 & \ldots, A_2(x), \ldots & \rightsquigarrow & \ldots, A_1(x), \ldots \\
\exists P \sqsubseteq A & \ldots, A(x), \ldots & \rightsquigarrow & \ldots, P(x, \_), \ldots \\
\exists P^- \sqsubseteq A & \ldots, A(x), \ldots & \rightsquigarrow & \ldots, P(\_, x), \ldots \\
A \sqsubseteq \exists P & \ldots, P(x, \_), \ldots & \rightsquigarrow & \ldots, A(x), \ldots \\
A \sqsubseteq \exists P^- & \ldots, P(\_, x), \ldots & \rightsquigarrow & \ldots, A(x), \ldots \\
\exists P_1 \sqsubseteq \exists P_2 & \ldots, P_2(x, \_), \ldots & \rightsquigarrow & \ldots, P_1(x, \_), \ldots \\
P_1 \sqsubseteq P_2 & \ldots, P_2(x, y), \ldots & \rightsquigarrow & \ldots, P_1(x, y), \ldots \\
P_1 \sqsubseteq P_2^- & \ldots, P_2(x, y), \ldots & \rightsquigarrow & \ldots, P_1(y, x), \ldots \\
 & \vdots & &
\end{array}
$$

where '_' denotes an **unbound** variable, i.e., a variable that appears only once

continued on next slide

Query rewriting – the procedure (2\2)

…

- Find (if there are) two atoms of **q'** that **unify**, and apply the unifier to **q'**.

Each time, the result of the above steps is added to the queries to be processed.

Note: Unifying atoms can make rules applicable that were not so before, and is required for completeness of the method.

The process **stops** when neither of the previous two steps are applicable

The **UCQ** resulting from this process is the **perfect rewriting** $r_{q,T}$ of the input query w.r.t. the **TBox**

Query rewriting algorithm

Algorithm ***PerfectRef*****(Q; T_P)**

Input: union of conjunctive queries **Q**, set of DL-Lite$_A$ PIs T_P

Output: union of conjunctive queries ***PR***

PR := **Q**;

repeat

PR' := ***PR***;

for each $q \in PR'$ **do**

for each atom $g \in$ **q do**

for each PI $I \in T_P$ **do**

if I is applicable to g **then** ***PR*** := ***PR*** ∪ { ***ApplyPI***(**q**, g, I) };

for each pair of atoms $g_1, g_2 \in q$ **do**

if g_1 and g_2 unify then ***PR*** := ***PR*** ∪ { ***Reduce***(**q**, g_1, g_2) };

until ***PR'*** = ***PR***;

return ***PR;***

Observations: 1) **Termination** follows from having only finitely many different rewritings.

2) **NIs** or **functionalities** do not play **any role** in the rewriting of the query.

Perfect rewriting in DL-Lite – Example

T = { AssistantProf $\sqsubseteq$ Professor,
Professor $\sqsubseteq \exists$teaches,
$\exists$teaches$^{-} \sqsubseteq$ Course }

A = { teaches(john, math), AssistantProf(john),
teaches(bob,batabase), AssistantProf(mary) }

Query: $q : \{(x) \mid \exists y.\, teaches(x, y) \wedge Course(y)\}$

Perfect rewriting: $r_{q,T}$:

$\{(x) \mid \exists y.\, \boldsymbol{teaches}(x, y) \wedge \boldsymbol{Course}(y)\}$

$\{(x) \mid \exists y, s.\, \boldsymbol{teaches}(x, y) \wedge \boldsymbol{teaches}(s, y)\}$

$\{(x) \mid \exists y.\, \boldsymbol{teaches}(x, y)\}$

$\{(x) \mid \boldsymbol{Professor}(x))\}$

$\{(x) \mid \boldsymbol{AssistantProf}(x))\}$

Evaluating $r_{q,T}$ over the ABox A (seen as a DB) produces as **answer** {john, bob, mary}.

Query answering over satisfiable DL-Lite ontologies

For an **ABox A** and a query **q** over **A**, let $\mathbf{Eval_{cwa}(q,A)}$ denote the evaluation of **q** over **A** considered as a **database** (i.e., considered under the **CWA**).

Theorem: Let **T** be a **DL-Lite TBox**, $\mathbf{T_P}$ the set of **PIs** in **T** , and **q** a **CQ** over **T**. Then, for each **ABox A** such that $\langle \mathbf{T}, \mathbf{A} \rangle$ is **satisfiable**, we have that

$$\mathbf{cert(q, \langle T, A \rangle) = Eval_{cwa}(PerfectRef(q, T_P), A):}$$

As a consequence, query answering over a satisfiable **DL-Lite** ontology is **FOL-rewritable**.

Notice that we did not use **NIs** or **functionality** assertions of **T** in computing $\mathbf{cert(q, \langle T, A \rangle)}$. Indeed, when the ontology is satisfiable, we can ignore Nis and functionality assertions for query answering.

Using RDBMS technology for query answering

The **ABox A** can be stored as a **relational database** in a standard RDBMS:

- For each **atomic concept C** of the ontology define a **unary relational table tabC**, and populate it with each $\langle c \rangle$ such that $\mathbf{C}(c) \in \mathbf{A}$.
- For each **atomic role P** of the ontology, define a **binary relational table tabP**, and populate it with each $\langle c_1, c_2 \rangle$ such that $\mathbf{P}(c_1, c_2) \in \mathbf{A}$.

We have that query answering over satisfiable DL-Lite ontologies can be done effectively using RDBMS technology:

$$\mathbf{cert(q, \langle T, A \rangle) = Eval(SQL(PerfectRef(q, T_P)), DB(A))}$$

Where:

- **Eval(q,DB)** denotes the evaluation of an SQL query **q** over a database **DB**.
- **SQL(q)** denotes the SQL encoding of a UCQ **q**.
- **DB(A)** denotes the database obtained as above.

Satisfiability of ontologies with only Positive Inclusions

Let us now consider the problem of establishing whether an ontology is satisfiable.

A first notable result tells us that positive inclusion assertions alone cannot give rise to unsatisable ontologies.

Theorem: Let **O** = ⟨**T**, **A**⟩ be a DL-Lite ontology where **T** contains **only PIs.**

Then, **O** is satisable.

Unsatisability in DL-Lite$_A$ ontologies can be caused by NIs or by functionality assertions.

Checking satisfiability of DL-Lite$_A$ ontologies

Satisability of a **DL-Lite$_A$** ontology **O** = ⟨**T**, **A**⟩ is reduced to evaluating over **DB(A)** a query that asks for the existence of objects violating the **NIs** or **functionality** assertions.

Let **T$_P$** be the set of **PIs** in **T**. We deal with **NIs** and **functionality** assertions differently

- For each **NI** $N \in$ **T**, we construct a Boolean CQ **q$_N$** such that ⟨**T$_P$**, **A**⟩ ⊨ **q$_N$** iff ⟨**T$_P$**,∪ { N }, **A**⟩ is **unsatisfiable**.
- We check whether ⟨**T$_P$**, **A**⟩ ⊨ **q$_N$** by using **PerfectRef**, i.e., we compute **PerfectRef(q$_N$, T$_P$,)**, and evaluate it over **A** (considered as a database).

- For each **functionality assertion** $F \in$ **T**, we construct a Boolean FOL query **q$_F$** such that **A** ⊨ **q$_F$**, iff ⟨{F}, **A**⟩ is **unsatisfiable**.
- We check whether **A** ⊨ **q$_F$** by directly evaluating **q$_F$** over **A** (considered as a database).

Checking violations of negative inclusions

For each NI N in T we compute a boolean CQ q_N according to the following rules:

N		q_N
$A_1 \sqsubseteq \neg A_2$	$\Rightarrow$	$\{(\quad) \mid \exists x.\, A_1(x) \wedge A_2(x)\}$
$\exists P \sqsubseteq \neg A$ or $A \sqsubseteq \neg \exists P$	$\Rightarrow$	$\{(\quad) \mid \exists x, y.\, P(x,y) \wedge A(x)\}$
$\exists P^- \sqsubseteq \neg A$ or $A \sqsubseteq \neg \exists P^-$	$\Rightarrow$	$\{(\quad) \mid \exists x, y.\, P(x,y) \wedge A(y)\}$
$\exists P_1 \sqsubseteq \neg \exists P_2$	$\Rightarrow$	$\{(\quad) \mid \exists x, y, z.\, P_1(x,y) \wedge P_2(x,z)\}$
$\exists P_1 \sqsubseteq \neg \exists P^-_2$	$\Rightarrow$	$\{(\quad) \mid \exists x, y, z.\, P_1(x,y) \wedge P_2(z,x)\}$
$\exists P^-_1 \sqsubseteq \neg \exists P_2$	$\Rightarrow$	$\{(\quad) \mid \exists x, y, z.\, P_1(x,y) \wedge P_2(y,z)\}$
$\exists P^-_1 \sqsubseteq \neg \exists P^-_2$	$\Rightarrow$	$\{(\quad) \mid \exists x, y, z.\, P_1(x,y) \wedge P_2(z,y)\}$
$P_1 \sqsubseteq \neg P_2$ *or* $P^-_1 \sqsubseteq \neg P^-_2$	$\Rightarrow$	$\{(\quad) \mid \exists x, y.\, P_1(x,y) \wedge P_2(x,y)\}$
$P^-_1 \sqsubseteq \neg P_2$ *or* $P_1 \sqsubseteq \neg P^-_2$	$\Rightarrow$	$\{(\quad) \mid \exists x, y.\, P_1(x,y) \wedge P_2(y,x)\}$

Checking violations of functionality assertions

For each functionality assertions F in T we compute a Boolean FOL query q_F according to the following rules:

$\boldsymbol{functionality\ assertion}$		$\boldsymbol{q_F}$
$(\boldsymbol{funct\ P})$	$\Rightarrow$	$\{(\quad) \mid \exists x, y, z.\, P_1(x, y) \wedge P_2(\mathrm{x}, \mathrm{z}) \wedge y \neq z\}$
$(\boldsymbol{funct\ P^-})$	$\Rightarrow$	$\{(\quad) \mid \exists x, y, z.\, P_1(\mathrm{y}, x) \wedge P_2(\mathrm{z}, \mathrm{x}) \wedge y \neq z\}$

From satisfiability to query answering in DL-Lite$_A$

Lemma (Separation for DL-Lite$_A$): Let **O** = ⟨**T**, **A**⟩ be a **DL-Lite$_A$** ontology, and T_P the set of PIs in **T**. Then, **O** is **unsatisfiable** iff one of the following condition holds:

- (a) There exists a NI $N \in$ **T** such that ⟨**T**, **A**⟩ $\models q_N$
- (b) There exists a functionality assertion $F \in$ **T** such that **A** $\models q_F$.

(a) relies on the properties that NIs do not interact with each other, and that interaction between NIs and PIs is captured through **PerfectRef**.

(b) exploits the property that NIs and PIs do not interact with functionalities: indeed, no functionality assertion is contradicted in a DL-Lite$_A$ ontology, beyond those explicitly contradicted by the **ABox**.

Notably, to check ontology satisfiability, each NI and each functionality assertion can be processed individually.

FOL-rewritability of satisfiability in DL-Lite$_A$

From the previous lemma and the theorem on query answering for satisfiable **DL-Lite$_A$** ontologies, we get the following result.

Theorem: Let **O** = ⟨**T**, **A**⟩ be a **DL-Lite$_A$** ontology, and **T$_P$** the set of PIs in **T**. Then, **O** is **unsatisfiable** iff one of the following condition holds:

- (a) There exists a NI N 2 T s.t. **Eval$_{cwa}$(PerfectRef(q_N, T$_P$), A)** returns **true**.
- (b) There exists a func. assertion F 2 T s.t. **Eval$_{cwa}$**(q_F, **A**) returns **true**.

Note: All the queries **q$_N$** and **q$_F$** can be combined into a single UCQ. Hence, satisfiability of a **DL-Lite$_A$** ontology is reduced to evaluating a FOL-query over an ontology whose TBox consists of positive inclusions only (and hence is satisable).

Complexity of reasoning in DL-Lite$_A$

Theorem: **Query answering** over **DL-Lite$_A$** ontologies is

- **NP-complete** in the size of **query and ontology** (combined complexity).
- **PTime** in the size of the **ontology** (schema + data complexity).
- **AC^0** in the size of the **ABox** (data complexity).

Theorem: **Checking satisfiability** of **DL-Lite$_A$** ontologies is

- **PTime** in the size of the **ontology** (schema + data complexity).
- **AC^0** in the size of the **ABox** (data complexity).

Theorem: **TBox reasoning** over DL-Lite$_A$ ontologies is **PTime** in the size of the **TBox** (schema complexity).

Linking ontologies to relational data (Query answering in OBDM)

Managing ABoxes

In the traditional DL setting, it is assumed that the data is maintained in the **ABox** of the ontology:

- The **ABox** is perfectly compatible with the **TBox**:
  - the vocabulary of concepts, roles, and attributes is the one used in the **TBox**.
  - the **ABox** "stores" abstract objects, and these objects and their properties are those returned by queries over the ontology.

- There may be different ways to manage the **ABox** from a physical point of view:
  - DLs reasoners maintain the **ABox** is main-memory data structures.
  - when an **ABox** becomes large, managing it in secondary storage may be required, but this is again handled directly by the reasoner.

Data in external sources

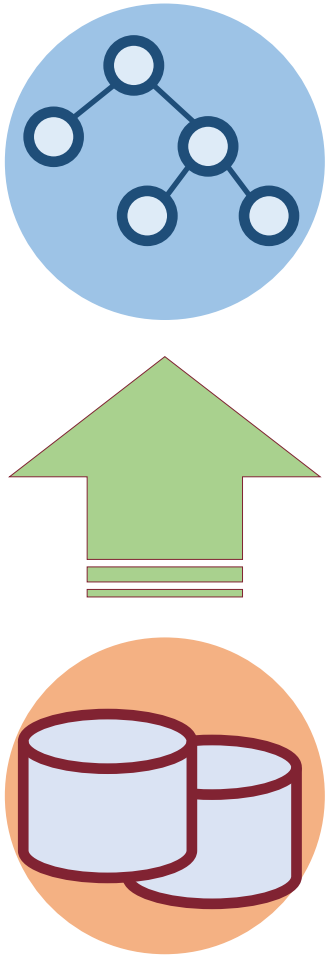
There are several situations where the assumptions of having the data in an **ABox** managed directly by the ontology system (e.g., a Description Logics reasoner) is not feasible or realistic:

- When the **ABox** is very large, so that it requires relational database technology.
- When we have no direct control over the data since it belongs to some external organization, which controls the access to it.
- When **multiple data sources** need to be accessed, such as in Information Integration.
- We would like to deal with such a situation by keeping the data in the external (relational) storage, and performing **query answering** by leveraging the capabilities of the **relational engine**.

Ontology-based data management: Architecture

The architecture of an **OBDM** system is based on three main components:

- **Ontology**: provides a unified, conceptual view of the managed information.
- **Data source(s)**: are external and independent (possibly multiple and heterogeneous).
- **Mappings**: semantically link data at the sources with the ontology.



The impedance mismatch problem

We have to deal with the **impedance mismatch problem**:

- Sources store data, which is constituted by values taken from concrete domains, such as strings, integers, codes, . . .
- Instead, instances of concepts and relations in an ontology are (abstract) objects.

Solution:

- We need to specify how to construct from the data values in the relational sources the (abstract) objects that populate the **ABox** of the ontology.
- This specification is embedded in the mappings between the data sources and the ontology.

Note: the **ABox** here is only **virtual**, and the objects are not materialized.

Solution to the impedance mismatch problem

We need to define a **mapping language** that allows for specifying how to transform data into abstract objects:

- Each mapping assertion maps:
  - a query that retrieves values from a data source to . . .
  - a set of atoms specified over the ontology.
- **Basic idea**: We need **constructors** to "create" ontology objects out of tuples of values in the database. Formally, such constructors can be **Skolem functions**.
- **Semantics of mappings**:
  - Objects are denoted by terms.
  - Different terms denote different objects (i.e., we make the **unique name assumption** on terms).

Impedance mismatch – Example

Suppose our data are store in a database containing information about employees and the projects they work in.

- D1[SSN:String; PrName:String] : Employees and projects they work for
- D2[Code:String; Salary:Int] : Employee's code with salary
- D3[Code:String; SSN:String] : Employee's Code with SSN

We know that:

- An employee is **identified** by her **SSN**.
- A project is **identified** by its **name**.

Intuitively:

- An object for an employee should be created from her SSN: **pers(SSN)**
- An objects for a project should be created from its name: **proj(PrName)**

Impedance mismatch: the technical solution

We need to associate the data in the tables to objects in the ontology.

- We introduce an alphabet Λ of **function symbols**, each with an associated arity.
- Let Γ_V be the alphabet of constants (values) appearing in the sources.
- To denote objects, i.e., instance of the concepts in the ontology, we use **object terms** instead of **object constants**.
- An **object term** has the form $\mathbf{f}(d_1, \ldots, d_n)$, with $\mathbf{f} \in \Lambda$, and each d_i a value constant in Γ_V .

Example

- If a person is identified by her **SSN**, we can introduce a function symbol $\mathbf{pers}_{/1}$. If **VRD56B25** is a **SSN**, then **pers(VRD56B25)** denotes a person.
- If a person is identified by her **name** and **dateOfBirth**, we can introduce a function symbol $\mathbf{pers}_{/2}$. Then **pers(Bob, 24/05/1941)** denotes a person.

Let's pick up from where we left off

In the context of ontology-based data management:

- Which is the "right" (user's) **query language** → **UCQ**
- Which is the "right" **ontology language** → **DL-Lite**
- How can we bridge the **semantic mismatch** between the ontology and the data sources? → We use **object terms** instead of **object constants**. To generate these terms we make use of **Skolem functions**.

Mapping assertions

Mapping assertions are used to extract the data from the DB to populate the ontology.

We make use of **variable terms**, which are like **object terms**, but with **variables** instead of **values** as arguments of the functions.

Definition: A **mapping assertion** between a database **D** and a TBox **T** has the form*

$$\Phi(\bar{x}) \rightsquigarrow \Psi(\bar{t}, \bar{y})$$

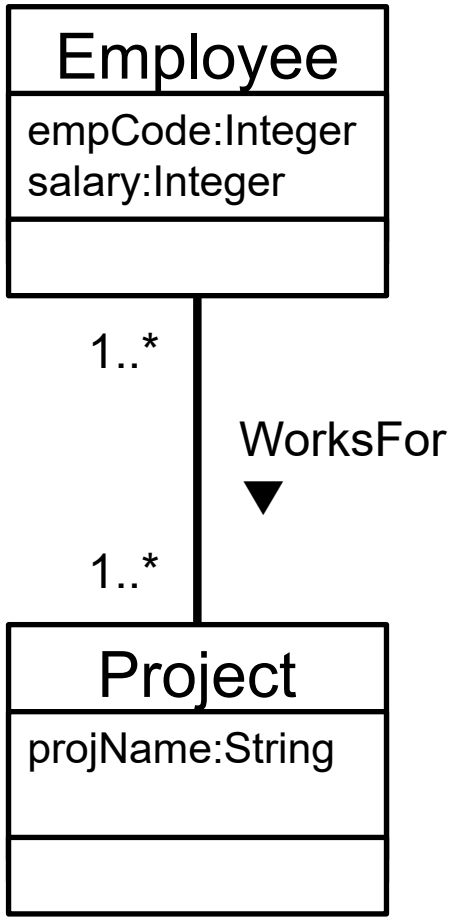
where:

- $\Phi(\bar{x})$ is an arbitrary SQL query of arity n > 0 over **D**;
- $\Psi(\bar{t}, \bar{y})$ is a conjunction of atoms whose predicates are **atomic concepts** and **roles** of **T**;
- $\bar{x}$ and $\bar{y}$ are variables, with $\bar{x} \subseteq \bar{y}$
- $\bar{t}$ are **variable terms** of the form $f(\bar{z})$, with $f \in \Lambda$ and $\bar{z} \subseteq \bar{x}$.

* **Note: this is a form of GAV mapping**

Mapping assertions – Example (1/2)

Consider the following TBox T (UML)

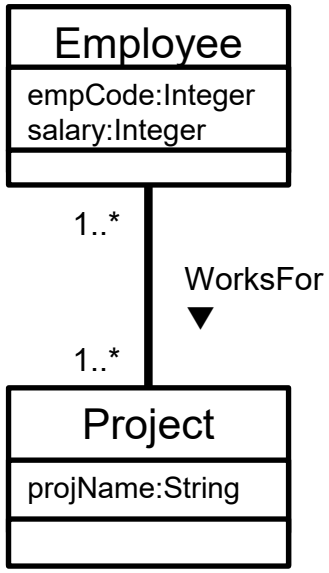


DL-Lite$_A$ axioms

$Employee \sqsubseteq \exists WorksFor$
$Project \sqsubseteq \exists WorksFor^-$
$\exists WorksFor \sqsubseteq Employee$
$\exists \mathrm{WorksFor}^- \sqsubseteq Project$
$Employee \sqsubseteq \delta(empCode)$
$Employee \sqsubseteq \delta(salary)$
$Project \sqsubseteq \delta(projName)$
$\delta(empCode) \sqsubseteq Employee$
$\delta(empCode) \sqsubseteq Employee$
$\delta(projName) \sqsubseteq Project$
$\rho(empCode) \sqsubseteq xsd{:}\,integer$
$\rho(salary) \sqsubseteq xsd{:}\,integer$
$\rho(projName) \sqsubseteq xsd{:}\,string$
$(funct\ empCode)$
$(funct\ salary)$
$(funct\ projName)$

Mapping assertions – Example (2/2)

TBox T (UML)



Source schema S

D1(SSN:string, PrName:string)
Employees and the project they work for

D2(Code:string, Salary:integer)
Employee's Code with salary

D3(SSN:string, Code:integer)
Employee's Code with SSN

…

Mapping M

m_1:
```
SELECT SSN, PrName
FROM D1
```
⇝
Eployee(**pers**(SSN)),
Project(**prj**(PrName)),
WorksFor(**pers**(SSN), **prj**(PrName)),
projName(**prj**(PrName), PrName)

m_2:
```
SELECT SSN, Code
FROM D3
```
⇝
Eployee(**pers**(SSN)),
empCode(**pers**(SSN), Code)

m_3:
```
SELECT SSN, Salary
FROM D2, D3
WHERE D2.Code = D3.Code
```
⇝
Eployee(**pers**(SSN)),
Salary(**pers**(SSN), Salary)

Ontology-Based Data Management: Formalization

To formalize OBDM, we distinguish between the **intensional** and the **extensional** level information.

Definition: An **OBDM specification** is a triple $\mathbf{P} = \langle \mathbf{T}, \mathbf{M}, \mathbf{S} \rangle$, where:

- **T** is a **DL TBox** providing the intensional level of an ontology.
- **S** is a (possibly federated) relational database schema for the **data sources**, possibly with constraints;
- **M** is a set of **mapping assertions** between **T** and **S**.

Definition: An **OBDM instance (or system)** is a pair $\mathbf{O} = \langle \mathbf{P}, \mathbf{D} \rangle$, where:

- $\mathbf{P} = \langle \mathbf{T}, \mathbf{M}, \mathbf{S} \rangle$ is an **OBDM specification**, and
- **D** is a **relational database** for **S**.

Semantics of an OBDM instance: Intuition

Given an **OBDM instance**, the **mapping M** encodes how the data **D** in the source(s) **S** should be used to populate the elements of the **TBox T**.

The data **D** and the mapping **M** define a **virtual data layer V**, which behaves like a **(virtual) ABox**.

Queries are answered w.r.t. **T** and **V**.

⟹ We want to **avoid materializing** the data of **V**. Instead, the intensional information in **T** and **M** is used to **translate** queries over **T** into queries formulated over **S**.

OBDM vs. Query Answering in Ontologies (QAO)

- **OBDM** relies on **QAO** to process queries w.r.t. the **TBox T**, but in addition is concerned with **efficiently** dealing with the **mapping M**.

Semantics of mappings

To define the semantics of an OBDM instance **O** = ⟨**P**, **D**⟩ with **P** = ⟨**T**, **M**, **S**⟩, we first need to define the semantics of mappings.

Definition: Satisfaction of a mapping assertion with respect to a database

An interpretation I satisfies a mapping assertion $\Phi(\overline{x}) \rightsquigarrow \Psi(\overline{t}, \overline{y})$ in **M** with respect to a database **D**, if for each tuple of values $\overline{v} \in \boldsymbol{Eval}(\Phi, \boldsymbol{D})$, and for each ground atom in $\Psi[\overline{x} \backslash \overline{v}]$, we have that:

- if the ground atom is $A(s)$, then $s^I \in A^I$.
- if the ground atom is $P(s_1, s_2)$, then $(s_1^I, s_2^I) \in P^I$

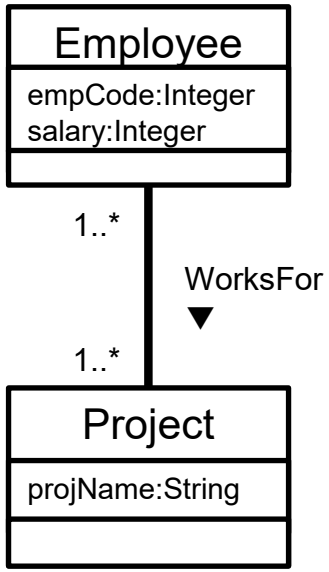
Intuitively, I satisfies $\Phi \rightsquigarrow \Psi$ w.r.t. **D** if all facts obtained by evaluating Φ over **D** and then propagating the answers to Ψ, hold in I.

Note: $\boldsymbol{Eval}(\Phi, \boldsymbol{D})$ denotes the result of evaluating Φ over the database **D**.

$\Psi[\overline{x} \backslash \overline{v}]$ denotes Ψ where each x_i has been substituted with v_i.

Semantics of mappings – Example

TBox T (UML)



Source database D

D_1

SSN	PrName
23AB	Apollo

D_2

Code	Salary
e23	15000

D_3

SSN	Code
23AB	e23

Mapping M

m_1: `SELECT SSN, PrName FROM D1` ⇝ Eployee(**pers**(SSN)), Project(**prj**(PrName)), WorksFor(**pers**(SSN), **prj**(PrName)), projName(**prj**(PrName), PrName)

The following interpretation satisfies m_1 with respect to D (note that we are directly using **object terms** as domain elements):

$$\boldsymbol{I}: \boldsymbol{\Delta_O^I} = \{\boldsymbol{pers}(23AB), \boldsymbol{prj}(Apollo), \dots\},\ \boldsymbol{\Delta_V^I} = \{Apollo, 15000, \dots\},$$
$$Employee^I = \{\boldsymbol{pers}(23AB), \dots\},$$
$$Project^I = \{\boldsymbol{prj}(Apollo), \dots\},$$
$$WorksFor^I = \{(\boldsymbol{pers}(23AB), \boldsymbol{prj}(Apollo)), \dots\},$$
$$projName^I = \{((\boldsymbol{prj}(Apollo), Apollo), \dots)\}, \dots$$

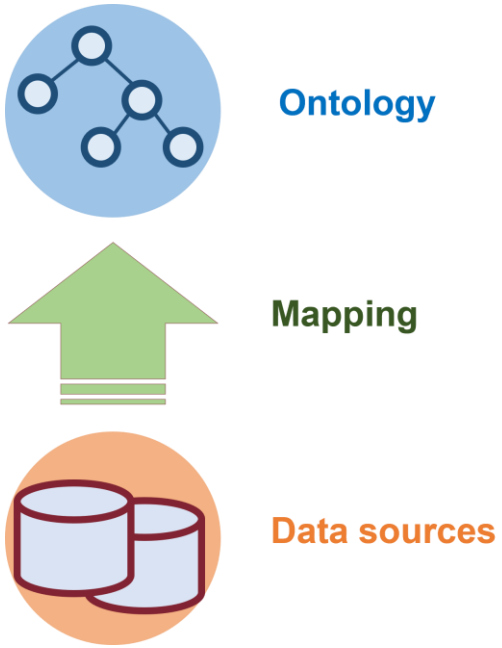
Semantics of an OBDM instance

Definition: Model of an OBDM instance

An interpretation $\mathcal{I}$ is a **model** of an OBDM instance $\mathbf{O} = \langle \mathbf{P}, \mathbf{D} \rangle$ with $\mathbf{P} = \langle \mathbf{T}, \mathbf{M}, \mathbf{S} \rangle$, if:

- $\mathcal{I}$ is a model of **T**, and
- $\mathcal{I}$ satisfies **M** w.r.t. **D**, i.e., $\mathcal{I}$ satifises every mapping assertion in **M** w.r.t. **D**.

An OBDM instance **O** is **satisfiable** if it admits at least one **model**.



Answering query over an OBDM system

Consider an OBDM instance $\mathbf{O} = \langle \mathbf{P}, \mathbf{D} \rangle$ with $\mathbf{P} = \langle \mathbf{T}, \mathbf{M}, \mathbf{S} \rangle$:

- Queries are posed over the TBox **T** .
- The data needed to answer queries is stored in the database **D**, which is compliant to **S**.
- The mapping **M** is used to bridge the gap between **T** and **D**.

Two approaches are possible:

- **bottom-up approach** (materialization approach): simpler, but typically less efficient
- **top-down approach** (virtualization approach): more sophisticated, but also more efficient

Both approaches require to first split the TBox queries in the mapping assertions into their constituent atoms (in order to obtain **"*pure*" GAV Mapping**).

Splitting of Mapping

A mapping assertions $\Phi(\bar{x}) \rightsquigarrow \Psi(\bar{t}, \bar{y})$ where the query $\Psi(\bar{t}, \bar{y})$ is constituted by the atoms $\alpha_1, \ldots, \alpha_k$, can be split into several mapping assertions:

$$\Phi \rightsquigarrow \alpha_1, \ldots, \Phi \rightsquigarrow \alpha_k$$

This can be done, since Ψ does **not contain existential variables**

Example:

m_1: `SELECT SSN, PrName FROM D1` $\rightsquigarrow$ Eployee(**pers**(SSN)), Project(**prj**(PrName)), WorksFor(**pers**(SSN), **prj**(PrName)), projName(**prj**(PrName), PrName)

is splitted into:

$m_{1\text{-}1}$: `SELECT SSN, PrName FROM D1` $\rightsquigarrow$ Eployee(**pers**(SSN))

$m_{1\text{-}2}$: `SELECT SSN, PrName FROM D1` $\rightsquigarrow$ Project(**prj**(PrName))

$m_{1\text{-}3}$: `SELECT SSN, PrName FROM D1` $\rightsquigarrow$ WorksFor(**pers**(SSN), **prj**(PrName))

$m_{1\text{-}4}$: `SELECT SSN, PrName FROM D1` $\rightsquigarrow$ projName(**prj**(PrName), PrName)

Bottom-up approach to query answering

Consider an OBDM instance **O** = ⟨**P**, **D**⟩ with **P** = ⟨**T**, **M**, **S**⟩:

The approach consists in a straightforward application of the mappings in order to compute a **materialized ABox**:

- Propagate the data from **D** through **M**, **materializing** an **ABox** $A_{M,D}$ (the constants in such an ABox are **values** and **object terms**).
- Apply to $A_{M,D}$ and to the TBox **T** , the **satisfiability** and **query answering** algorithms developed for **DL-Lite$_A$**.

This approach has several drawbacks:

- The technique is no more AC_0 in the data complexity, since the ABox $A_{M,D}$ to materialize is in general **polynomial** in the size of the data.
- $A_{M,D}$ may be very large, and thus it may be infeasible to actually materialize it.
- **Freshness** of $A_{M,D}$ with respect to the underlying data source(s) may be an issue.

Top-down approach to query answering

Given an OBDM instance **O** = ⟨**P**, **D**⟩ with **P** = ⟨**T**, **M**, **S**⟩ and a UCQ **q**, this approach consists of three steps:

1. **Reformulation**: Compute the **perfect rewriting** q_{pr} = **PerfectRef(q, T)** of the query q, using the assertions of the TBox **T**. As we know, the perfect rewriting q_{pr} is such that **cert(q,** ⟨**T**, **A**⟩**)** = **Eval$_{cwa}$(**q_{pr}**, A)** for each **ABox A**.

2. **Unfolding**: Compute from q_{pr} a new query q_{unf} by **unfolding** q_{pr} using (the split version of) the mappings **M** (as for GAV information integration systems)
   - Essentially we are going to replace each atom in q_{pr} with the corresponding query Φ over the database **D** in **M** - in fact **M** is a GAV mapping so the predicates of **T** (and hence of q_{pr}) are defined in **M** as views.
   - The query q_{unf} is such that **Eval(**q_{unf}**,D) = Eval$_{cwa}$(**q_{pr}**,** $A_{M,D}$**)** for each database **D**.

3. **Evaluation**: **Delegate** the evaluation of q_{unf} to the relational **DBMS** managing **D**.

Computational complexity of query answering

From the top-down approach to query answering, and the complexity results for DL-Lite, we obtain the following result

Theorem: In a DL-Lite OBDM instance $\mathbf{O} = \langle \mathbf{P}, \mathbf{D} \rangle$ with $\mathbf{P} = \langle \mathbf{T}, \mathbf{M}, \mathbf{S} \rangle$ query answering (i.e., the recognition problem) of a UCQ **q** is:

- **NP-complete** in the size of the query **q**.
- **PTime** in the size of the **TBox T** and the **mappings M**.
- $\mathbf{AC_0}$ in the size of the database **D**.

Note: The AC_0 result is a consequence of the fact that query answering in such a setting can be reduced to evaluating an SQL query over the relational database **D**.

Credits

The presentations of this module of the course are based on the original slides of Prof. Giuseppe De Giacomo, Prof. Diego Calvanese, Prof. Marco Console, Prof. Maurizio Lenzerini, and Prof. Domenico Lembo.